TEXT PREPROCESSING IN NLP

Complete walkthrough: 11 steps, one running example, and the reason behind every step

1. What is text preprocessing, and why do we need it?

Raw text is messy. Real reviews, tweets, chats and web pages contain capital letters, HTML tags, links, slang, typing mistakes and emojis. A machine learning model cannot read language the way humans do; it only counts and compares tokens. If "Movie", "movie" and "movie!" look like three different words, the model splits its knowledge between them and learns noise instead of meaning. **Text preprocessing is the cleaning and standardising of raw text before it reaches the model.** The result is a smaller vocabulary, less noise, faster training and better accuracy. The rule of thumb is *garbage in, garbage out*.

Golden rule: not every step is compulsory. Use common sense and choose steps by project need. Sentiment analysis of tweets: convert emojis and keep "!". Search engine: remove stop words and stem. Grammar tagging: keep almost everything.

01 Lower	02 HTML	03 URLs	04 Punct.	05 Chat	06 Spelling	07 Stop	08 Emojis	09 Token	10 Stem	11 Lemma
CLEANING (1-8): remove noise, fix text								SPLIT	NORMALISE (10-11)	

Three phases: **Cleaning (1-8)** removes noise and repairs the text. **Tokenization (9)** turns one long string into countable units. **Normalisation (10-11)** merges different forms of one word (run, running, ran) so the model sees a single idea. Each step block below answers four questions: what it is, why it is needed, where it is used, and what to watch out for.

2. Our running example (random review text)

The original tutorial uses the IMDB movie-review dataset (50,000 reviews). For these notes we take **one short random review** that contains every kind of noise, so each step has something to fix. Every step below takes the output of the previous step as its input. Yellow bold text shows what changed. "[emoji]" stands for a heart-eyes emoji.

RAW INPUT

```
<p>OMG!!! I just watched Inception yesterday [emoji] IMHO it was a GR8 movie, btw see https://www.imdb.com/inception
for details. The actors were runing, fighting and dreaming in the film!</p>
```

3. The step-by-step walkthrough

01 LOWERCASING

make every word look the same

What it is: Converts every capital letter in the text into a small letter.

Why it is required: A computer sees "Word" and "word" as two different strings. This inflates the vocabulary and splits word counts. Lowercasing merges them and also makes later dictionary look-ups (stop words, slang, stemmer) simple because those lists are stored in lowercase.

Where it is used: Sentiment analysis, text classification, spam detection, search engines, bag-of-words and TF-IDF features.

Watch out: Case sometimes carries meaning: "US" vs "us", or "Apple" the company vs "apple" the fruit. Skip it for named-entity recognition.

MINI EXAMPLE

```
Inception -> inception
IMHO -> imho
GR8 -> gr8
```

RUNNING TEXT AFTER THIS STEP

```
<p>omg!!! i just watched inception yesterday
[emoji] imho it was a gr8 movie, btw see
https://www.imdb.com/inception for details.
the actors were runing, fighting and dreaming
in the film!</p>
```

02 REMOVE HTML TAGS

strip the web-page wrapping

What it is: Deletes markup such as <p>,
 or that surrounds text scraped from websites.

Why it is required: Tags are formatting instructions, not language. If left in, "p" and "br" become very frequent but meaningless tokens that mislead the model and pollute word counts.

Where it is used: Web scraping, IMDB and Amazon reviews, news crawlers, forum posts, email bodies.

Watch out: Remove only the tag, keep the text inside it. Special codes such as & need separate decoding.

MINI EXAMPLE

```
<p>omg!!! ... film!</p>
-> omg!!! ... film!
```

RUNNING TEXT AFTER THIS STEP

```
omg!!! i just watched inception yesterday
[emoji] imho it was a gr8 movie, btw see
https://www.imdb.com/inception for details.
the actors were runing, fighting and dreaming
in the film!
```

03 REMOVE URLS

delete links that add no meaning

What it is: Finds web addresses beginning with http, https or www and deletes them.

Why it is required: A link is a unique string that appears once and never again. It adds thousands of one-off tokens and tells nothing about opinion or topic. Do this **before** punctuation removal, otherwise the link breaks into pieces ("https", "www", "imdb", "com") that can no longer be recognised as a link.

Where it is used: Tweets, social media, product reviews, emails, news comments. In spam detection the number of links may be kept as a feature instead.

Watch out: A cleaning rule must match the whole address, so test it on sample links before applying it to the full dataset.

MINI EXAMPLE

```
see https://www.imdb.com/inception for
details
-> see for details
```

RUNNING TEXT AFTER THIS STEP

```
omg!!! i just watched inception yesterday
[emoji] imho it was a gr8 movie, btw see for
details. the actors were runing, fighting and
dreaming in the film!
```

04 REMOVE PUNCTUATION

keep the words, drop the symbols

What it is: Removes symbols such as ! , ? " ' ( ) : ; from the text. In these notes the full stop is kept so that sentence boundaries survive for sentence tokenization.

Why it is required: Punctuation sticks to words, so "movie," and "movie" become different tokens. Removing it unifies them and cuts noise.

Where it is used: Sentiment analysis, topic modelling, text classification, bag-of-words models.

Watch out: "!!!" can show strong emotion in sentiment work, and chatbots, translation and grammar tagging need punctuation. "don't" becomes "dont".

MINI EXAMPLE

```
omg!!! -> omg
movie, -> movie
runing, -> runing
```

RUNNING TEXT AFTER THIS STEP

```
omg i just watched inception yesterday [emoji]
imho it was a gr8 movie btw see for details.
the actors were runing fighting and dreaming
in the film
```

05 CHAT WORD TREATMENT

expand internet slang

What it is: Replaces slang and short forms (IMHO, BTW, GR8, FYI, LOL, BRB) with full forms using a look-up dictionary of "slang = meaning" pairs.

Why it is required: Models trained on formal text do not know "gr8" or "imho". Expanding them brings the meaning into normal vocabulary, so "gr8" now counts as the positive word "great". It is done **before** spelling correction, otherwise the corrector may "fix" slang into the wrong word.

Where it is used: Twitter, WhatsApp and Instagram analysis, customer-support chats, chatbots, comment moderation.

Watch out: The dictionary is hand-made and never complete, so add your own entries (we added OMG). Some short forms are ambiguous ("U", "TIME"). Matching must ignore case.

MINI EXAMPLE

```
omg -> oh my god
imho -> in my honest opinion
gr8 -> great
btw -> by the way
```

RUNNING TEXT AFTER THIS STEP

```
oh my god i just watched inception yesterday
[emoji] in my honest opinion it was a great
movie by the way see for details. the actors
were runing fighting and dreaming in the film
```

06 SPELLING CORRECTION

fix typing mistakes

What it is: Detects misspelled words and replaces each with the most probable correct word, using a probability-based tool such as TextBlob.

Why it is required: "runing" and "running" would be counted as two words, and the typo is so rare that the model learns nothing from it. Fixing it gathers all evidence under one word. Stemming and lemmatization also need real words to work well.

Where it is used: User-typed text, social posts, search queries, OCR of scanned pages, speech-to-text output, survey answers.

Watch out: It is slow on big datasets and can wrongly change names, brands and technical terms. Check a sample of its changes first.

MINI EXAMPLE

```
runing -> running
(texxt -> text, curciaal -> crucial)
```

RUNNING TEXT AFTER THIS STEP

```
oh my god i just watched inception yesterday
[emoji] in my honest opinion it was a great
movie by the way see for details. the actors
were running fighting and dreaming in the film
```

07 REMOVING STOP WORDS

drop the filler words

What it is: Deletes very common function words such as the, is, a, in, was, for, my, by and, which hold a sentence together but add little meaning.

Why it is required: Stop words make up nearly half of all running text yet say almost nothing about topic or opinion. Removing them shrinks the data, speeds up training and lets the important content words stand out in frequency methods like TF-IDF.

Where it is used: Search engines, keyword extraction, topic modelling, text classification, word clouds, document similarity.

Watch out: Do NOT use it for grammar tagging, translation or question answering. Most lists contain "not" and "no", so "not good" becomes "good" and flips the sentiment. Edit the list.

MINI EXAMPLE

```
i just watched -> watched
by the way -> way
(i, just, my, by, the, was removed)
```

RUNNING TEXT AFTER THIS STEP

```
oh god watched inception yesterday [emoji]
honest opinion great movie way see details.
actors running fighting dreaming film
```

08 HANDLING EMOJIS

remove or translate pictures

What it is: Emojis are Unicode picture symbols. They can be removed completely, or replaced by short words that describe them.

Why it is required: Many tokenizers and models cannot read emoji and turn them into broken characters. Yet emoji carry strong feeling, so the choice depends on the task: remove them for neutral work, translate them for emotion work.

Where it is used: Twitter and Instagram sentiment, WhatsApp chat analysis, brand monitoring, product reviews.

Watch out: Deleting the emoji in a sentiment project throws away a clear positive signal. Translate it first (heart-eyes becomes "love"). In our example we remove it.

MINI EXAMPLE

```
yesterday [emoji] honest -> yesterday honest
(or heart-eyes -> "love")
```

RUNNING TEXT AFTER THIS STEP

```
oh god watched inception yesterday honest
opinion great movie way see details. actors
running fighting dreaming film
```

CLEANING CHECKPOINT (end of steps 1-8): the noisy review has lost its HTML, link, symbols, slang, typo, filler words and emoji. What remains is a short, clean string of content words, ready to be split into tokens and normalised in steps 9-11.

09 TOKENIZATION

cut the text into pieces

What it is: Splits text into smaller units called tokens. Word tokenization gives words (and punctuation marks) one by one; sentence tokenization gives whole sentences.

Why it is required: A model cannot read a long string. It works on units that can be counted, listed in a vocabulary and turned into numbers. Tokenization is the doorway to every numeric method and is needed before word-by-word stemming or lemmatization.

Where it is used: Every NLP task: bag-of-words, TF-IDF, word embeddings, search indexing, translation, chatbots. Sentence splitting helps summarization and translation.

Watch out: Splitting on spaces alone is not enough ("New York", "don't", "Dr."), so use a trained tokenizer.

MINI EXAMPLE

```
WORDS: [oh] [god] [watched] [inception]
[yesterday] [honest] [opinion] [great]
[movie] [way] [see] [details] [.] [actors]
[running] [fighting] [dreaming] [film]
```

RUNNING TEXT AFTER THIS STEP

18 word tokens (17 words + 1 full stop) and **2 sentences**:

```
S1: oh god watched ... see details.
S2: actors running fighting dreaming film
```

10 STEMMING

chop endings, keep the root

What it is: Reduces words to a root (stem) by cutting suffixes with fixed rules, so walk, walks, walking and walked all become "walk". The stem need not be a real word. The Porter stemmer is the classic choice.

Why it is required: It reduces word forms to one shared base so related words are counted together. This shrinks the vocabulary again and is very fast.

Where it is used: Search engines and information retrieval (searching "running" also finds "run"), document classification, spam filters, large datasets where speed matters.

Watch out: Rules are blunt. They may over-cut: "movie" becomes "movi" and "inception" becomes "incept". Output can be hard to read.

MINI EXAMPLE

```
watched -> watch
details -> detail
running -> run
movie -> movi
inception -> incept
```

RUNNING TEXT AFTER THIS STEP

```
oh god watch incept yesterday honest opinion
great movi way see detail . actor run fight
dream film
```

11 LEMMATIZATION

reduce to the true dictionary word

What it is: Reduces a word to its lemma, the proper dictionary form, using a vocabulary and the word's part of speech. The output is always a real word.

Why it is required: It does what stemming does but without breaking words, so results stay readable and accurate. It also handles irregular forms that stemming cannot: "was" becomes "be", "better" becomes "good", "mice" becomes "mouse".

Where it is used: Chatbots, machine translation, question answering, summarization, sentiment analysis. Use it when meaning matters more than speed.

Watch out: It is slower than stemming and needs the part of speech: "running" as a verb gives "run", as a noun it stays "running".

MINI EXAMPLE

```
watched -> watch
details -> detail
actors -> actor
movie -> movie
inception -> inception
```

RUNNING TEXT AFTER THIS STEP

FINAL CLEAN TEXT: `oh god watch inception yesterday honest opinion great movie way see detail . actor run fight dream film`

4. Stemming vs lemmatization, and choosing steps by task

Point	Stemming	Lemmatization
Method	Fixed rules cut word endings	Dictionary plus grammar (part of speech)
Output	May be a non-word (movi, incept)	Always a real word (movie, inception)
Speed / accuracy	Very fast, rougher	Slower, more accurate
Best for	Search, big data, quick models	Chatbots, translation, meaning-heavy tasks

Task	Keep / skip
Sentiment	Translate emojis, expand slang, keep "!", keep "not"
Search engine	Lowercase, drop stop words, stem
Grammar tagging	Keep case, stop words, punctuation
Chatbot	Expand slang, fix spelling, lemmatize
Topic modelling	Drop stop words and punctuation, lemmatize

Order matters: (a) lowercase first so every later list matches; (b) remove URLs before punctuation or links shatter; (c) expand slang before spelling correction; (d) tokenize before stemming or lemmatizing. Result: messy HTML became 18 clean tokens.

5. One-line memory aid

01 Lower	Same word, same shape.	**02 HTML**	Tags are not language.	**03 URLs**	Links are one-off noise.
04 Punct.	Symbols stick to words.	**05 Chat**	Teach the model slang.	**06 Spell**	One word, one count.
07 Stop	Drop filler, keep "not".	**08 Emoji**	Remove or translate.	**09 Token**	Text becomes units.
10 Stem	Fast, crude root.	**11 Lemma**	True dictionary word.		